SwartzNet

Full-Text Search over the Mainline BitTorrent Network

The SwartzNet Project

July 2026

Abstract. BitTorrent moves more data than almost any other peer-to-peer system, yet it has no native way to *find* anything: discovery happens on centralized web indexes that are out-of-band, fragile, and a single point of censorship. A purely peer-to-peer search layer would let content be located directly by the same swarm that serves it, with no coordinating server. The central problem is that a search overlay normally demands its own protocol — a new handshake bit, a new DHT query, a new port — which fragments the network and makes the overlay trivial to fingerprint and block. SwartzNet solves this by carrying search entirely inside protocol surfaces that mainline clients already speak: the extension protocol, the DHT's signed-item store, and ordinary torrents. A vanilla client sees nothing but standard traffic. Search indexes are signed by their publishers, reconciled between peers with a rateless set protocol, and defended against spam by reputation and admission control rather than by a central authority. The result is a distributed full-text index that is indistinguishable, on the wire, from the network it lives in.

1. Introduction

Finding a torrent means leaving the network that hosts it. A user searches a web index, copies a magnet link, and only then rejoins the swarm. Those indexes are centralized: they are operated by identifiable parties, they are the point where takedowns and surveillance are applied, and when one disappears its catalogue disappears with it. The swarm itself — millions of nodes that already store and exchange the content — holds no searchable record of what it carries.

What is needed is a search layer that is part of the network rather than beside it: any node can publish what it indexes, any node can query its peers and the DHT, and no node is required for the system to function. The difficulty is doing this without changing what the network *is*. BitTorrent's reach comes from a shared set of standards (the BEP series); a search system that introduces a new reserved bit or a new DHT verb creates a second, incompatible network and paints a target on every participant. SwartzNet's design goal is therefore not merely "peer-to-peer search" but *invisible* peer-to-peer search: full-text discovery that reuses existing standards so completely that a node running it is, to any observer, a normal BitTorrent client that happens to be unusually well-connected.

2. Design Constraint: Mainline Compatibility

Every mechanism in SwartzNet is bound by one rule: a vanilla client must see only standard traffic. Concretely, SwartzNet adds **no new reserved handshake bit, no new DHT query, and no new UDP port.** It builds only on established extension points:

- **BEP 3/5/9/10** — the core protocol, the mainline DHT, metadata exchange, and the extension protocol (LTEP), which lets peers negotiate named sub-protocols inside the normal handshake.
- **BEP 44** — the DHT's store for small signed (mutable) and content-addressed (immutable) items, the substrate for a keyword index.
- **BEP 46** — mutable-torrent pointers in the DHT, reused as a stable pointer to a publisher's latest index snapshot.

- **BEP 51** — DHT infohash sampling, reused to discover what the network holds.

Because SwartzNet's peer-wire protocol is negotiated through LTEP, a peer that does not advertise it is never sent a SwartzNet message; because its DHT records are ordinary BEP-44 items, any client can read them and none is required to understand them. Compatibility is not a feature bolted on afterward — it is the frame every other decision is made inside. Any change that would let a vanilla client observe non-standard traffic is, by definition, a defect.

3. Identity and Signed Records

Search is only useful if results can be attributed and trusted, but a network with no accounts cannot rely on identity being *assigned*. Each node instead generates a persistent ed25519 keypair the first time it runs. That key is the node's publisher identity: it signs the records the node contributes to the shared index, and its reputation — good or bad — accrues to the key, not to an address.

A record is the atomic unit of the index: a signed statement that a given keyword applies to a given infohash, at a given time, optionally carrying a small proof-of-work stamp. Signing is designed so that **the infohash is never disturbed.** When a publisher signs a `.torrent`, the signature lives in top-level fields outside the info-dictionary, so the content's identity is unchanged and a vanilla client downloads it exactly as before. The signature authenticates *who is making a claim about* a torrent, never the torrent itself. This separation — stable content identity, attributable claims about it — is what lets an untrusted, open network accumulate an index whose entries can still be weighed by their source.

4. Layer L: The Local Index

The foundation is entirely local. Every node maintains a full-text index of the torrents it holds, built with an embedded search engine over the torrent names and — where the node chooses to index content — the text extracted from the files themselves (PDF, EPUB, office documents, plain text, subtitles). This layer answers queries instantly and offline; it is the ground truth a node can vouch for because it indexed the bytes directly. Content extraction is opt-in per torrent, and publication to the wider network can be turned off entirely, so the operator controls both what text a node indexes and whether the node announces anything beyond itself. The local index is deliberately isolated: it knows nothing of the wire protocols above it, and the layers above it never reach into its internals. Search results from different layers are kept separate and reconciled only at the presentation edge, never merged into a single ranked list that would blur which layer — and which trust model — produced a hit.

5. Layer S: Peer-Wire Search

The first way a node's index reaches others is directly, over the peer connections it already has. SwartzNet registers a named extension, `sn_search`, through LTEP. Immediately after the extension handshake, a participating peer sends a capability announcement — a mask of which search services it offers.

The opt-in is enforced on the *send* side, and structurally. When a node observes a peer announce `sn_search`, it mints an unforgeable capability token for that peer; sending any SwartzNet frame to a peer requires such a token, and there is no way to construct one except from an observed announcement. A peer that announces nothing is therefore never sent a frame — the silence is a property of the code paths, not a runtime check that could be bypassed or a policy that could be misconfigured. Queries themselves carry a search term and a scope, not a token; a responder answers according to its own sharing setting, a per-peer rate limit, and a ban list, and is free to answer a peer it has not itself queried. Results carry infohashes with names,

sizes, and seeder counts. Because the whole exchange rides inside an already-established peer connection, it needs no new socket and is invisible to peers who did not opt in.

6. Layer D: The DHT Keyword Index

Peer-wire search only reaches nodes one is already connected to. To make an index entry discoverable network-wide, SwartzNet publishes it into the DHT as a BEP-44 mutable item. A keyword is mapped to a DHT address by salting the publisher's key with the keyword, so each publisher owns an independent namespace for every term and no two publishers collide. A lookup for a term fetches, from each known publisher, the signed list of infohashes that publisher associates with it, then merges and scores those lists by publisher reputation.

BEP-44 items are small — on the order of a kilobyte — and expire after a couple of hours. SwartzNet works within both limits: a publisher re-announces on a timer, and when a keyword's entry would overflow the size cap the oldest hit is evicted rather than the record being dropped or a non-standard multi-item scheme being introduced. The keyword index therefore rides the standard signed-item store exactly as any other BEP-44 application would, and a vanilla client can read any of it. What SwartzNet adds is not a new DHT behaviour but a *convention* for what the signed bytes mean.

7. Set Reconciliation

Two nodes that have each accumulated many signed records will overlap heavily but not exactly. Naively exchanging full record sets to find the difference wastes bandwidth proportional to what both already share. SwartzNet instead reconciles record sets using a rateless invertible protocol (RIBLT): each side streams coded symbols computed over its records, and the *difference* between the two sets is decoded from a number of symbols proportional to the size of that difference, not to the size of the sets. Neither side needs to know the difference size in advance — it keeps sending symbols until the other side can peel out the missing records. This lets peers converge on the union of their indexes cheaply, turning every peer connection into an opportunity to fill gaps in coverage. Reconciliation is itself carried as `sn_search` messages, so it inherits the same opt-in, mainline-invisible properties as ordinary queries.

8. The Aggregate Index

The per-keyword DHT index answers "who claims this term?" one publisher at a time. A richer query — a prefix scan, or a single authenticated snapshot of a publisher's whole catalogue — needs a compact, verifiable structure larger than a single DHT item can hold. SwartzNet's Aggregate index packs a publisher's signed records into a signed B-tree: a self-contained, deterministic file with a signed trailer, where every record carries its own signature and the whole tree carries a fingerprint over its contents. A reader can verify the trailer's signature before trusting anything, verify each record independently, and run authenticated prefix queries over the tree — every answer carrying its own signature — while the format's page structure leaves partial, on-demand reads open for a later client.

The tree is distributed as an ordinary torrent, and a small BEP-44 pointer — a "publisher-pointer" item at a fixed, publisher-independent salt — advertises its infohash together with the tree's fingerprint. The fixed salt means every publisher's pointer lives at an address derived the same way, so the DHT cannot be scanned for SwartzNet users by salt; the fingerprint in the pointer binds the pointer to a specific tree, so a reader who fetches the tree can confirm its bytes are exactly the ones the signed pointer vouched for. This is the general shape of the whole system: a signed, content-addressed artifact distributed as a normal torrent, located through a standard signed pointer. The Aggregate index ships off by default — the per-keyword index

is the shipping path — but rides the same seams, so enabling it changes a mode, not the protocol.

9. Spam Resistance

An open index with no gatekeeper invites poisoning: a hostile publisher can claim any keyword for any infohash. SwartzNet does not try to prevent claims; it makes them cheap to *discount*. Four mechanisms compose:

1. **Reputation.** Each publisher key accrues a Bayesian-smoothed score from the outcomes of the hits it has returned — confirmed good, or flagged bad — so a key that produces useful results is weighted up and a key that produces junk is weighted down. Smoothing means a handful of flags cannot instantly bury a key, and a handful of fresh keys cannot instantly vouch for one.
2. **Deny-by-default admission.** A previously unknown publisher is not trusted merely because a few other unknown publishers endorse it — the classic Sybil move. New keys must clear a bar that fresh, mutually-endorsing identities cannot, or be seeded by an operator-provided anchor list.
3. **A known-good filter.** Confirmed-good infohashes are recorded in a compact Bloom filter, so results the local user (or its trusted set) has already vouched for can be surfaced preferentially at negligible cost.
4. **Optional proof-of-work.** A record may carry a hashcash stamp, letting a publisher spend computation to raise the cost of mass-producing claims. It is optional and bounded, a dial rather than a wall.

Crucially, none of these decisions are made by a model or a prompt; admission, reputation updates, and eviction are deterministic code with explicit thresholds and fail-closed defaults. The index tolerates bad actors instead of trying to exclude them, which is the only stance that survives in a permissionless network.

10. Capability Negotiation and Privacy

A node should share exactly what its operator intends and no more. SwartzNet splits its capability mask in two: an operator-controlled half (whether to answer queries at all, and whether to share file-name versus content hits) and a daemon-owned half (runtime facts such as whether the node is currently publishing). The two never mix, so an operator's "save my sharing preferences" can never accidentally toggle a runtime fact. A node also enforces its *current* sharing setting on every inbound query, so tightening what it is willing to answer takes effect at once. Unknown or future capability bits are tolerated, never rejected, so the mask can grow without a flag day. Because sharing is off unless announced and the send path is token-gated, the privacy-preserving default is also the mainline-compatible one: a node that shares nothing is wire-indistinguishable from a vanilla client.

11. Content Discovery

Two further mechanisms populate the index without a central catalogue. A node may publish a **companion content-index**: a compact, compressed snapshot of what it has indexed, distributed as an ordinary single-file torrent and advertised through a BEP-46 mutable pointer. Another node can *follow* a publisher, fetch its snapshots as they update, and import its catalogue — verifying authorship and de-duplicating across snapshots — so coverage spreads by subscription rather than by a server. Separately, a node can **crawl** the DHT using BEP-51 infohash sampling: a bounded, polite worker pool that samples infohashes from the nodes it reaches and expands its frontier from their neighbours, discovering what the network holds so it can be fetched, indexed, and re-published. Both mechanisms are opt-in and ride only standard verbs; neither downloads or indexes anything the operator did not ask for.

12. Conclusion

We have described a full-text search layer for BitTorrent that requires no coordinating server and no departure from the standards the network already runs on. Nodes index what they hold locally, answer each other over existing peer connections, publish signed keyword records into the DHT's standard signed-item store, reconcile those records with a rateless set protocol, and — optionally — distribute whole signed catalogues as ordinary torrents located through standard pointers. Trust is placed in cryptographic identity and earned reputation rather than in any authority, and spam is discounted by deterministic admission and scoring rather than excluded by a gatekeeper. The binding constraint throughout is mainline compatibility: every message rides an existing extension point, every record is a standard DHT item, and a node that shares nothing is invisible. The network needed no new protocol to become searchable — only a convention, signed and carried inside the one it already speaks.

References

1. B. Cohen, *The BitTorrent Protocol Specification* (BEP 3).
2. *DHT Protocol* (BEP 5); *Extension for Peers to Send Metadata Files* (BEP 9).
3. *Extension Protocol* (BEP 10).
4. A. Norberg et al., *Storing Arbitrary Data in the DHT* (BEP 44).
5. *Updating Torrents via DHT Mutable Items* (BEP 46).
6. *DHT Infohash Indexing* (BEP 51).
7. A. Back, *Hashcash — A Denial of Service Counter-Measure* (2002).
8. S. Nakamoto, *Bitcoin: A Peer-to-Peer Electronic Cash System* (2008).
9. L. Yang, Y. Gilad, M. Alizadeh, *Practical Rateless Set Reconciliation* (SIGCOMM 2024) — the rateless invertible-Bloom set-difference protocol.
10. SwartzNet architecture and protocol drafts: `docs/05-integration-design.md`, `docs/06-bep-sn_search-draft.md`, `docs/07-bep-dht-keyword-index-draft.md`.